MODULE 0 · INTRODUCTION

Lesson 0 — Introduction to JavaScript

Two parts: the core language on its own (Part 1), then jQuery — the DOM library Shiny's core and most Shiny extension packages are still built on (Part 2). Complete on its own, full examples for each idea.

Part 1 — Core JavaScript

0.1 — What JavaScript is, and where it runs

JavaScript started as a scripting language for web browsers and is now also a server runtime (Node.js). The *language* is identical in both places; what differs is the surrounding environment — a browser hands you `document` and `window` automatically, Node hands you `fs`, `process`, and no DOM at all. A Shiny custom-JS file runs in the browser environment specifically.

0.2 — Data types

JS has exactly 7 **primitive** types, plus one catch-all **reference** type:

```
string     "hello"          text
number     42, 3.14         one numeric type — int and float alike, no separate types
boolean    true / false
undefined  —                declared, never assigned
null       —                deliberately empty, assigned on purpose
bigint     10n              integers past Number.MAX_SAFE_INTEGER (~9 quadrillion)
symbol     Symbol()         rare; unique identifiers, mostly for advanced object tricks
object     {}, [], fn       everything else — copied by REFERENCE, not by value
```

`typeof x` tells you which one you're holding, with one wart worth memorizing: `typeof null` returns `"object"` — a 25-year-old bug baked permanently into the language, not a real object.

The primitive/reference split matters because of how assignment behaves. Primitives copy their value:

```
let a = 5;
let b = a;   // b gets a COPY of 5
b = 10;
console.log(a); // still 5 — independent
```

Objects and arrays copy a *pointer* to shared data, not the data itself:

```
let obj1 = { name: "draft" };
let obj2 = obj1;         // obj2 points to the SAME object, not a copy
obj2.name = "published";
console.log(obj1.name);  // "published" — same object, both changed
```

0.3 — Variables: const, let, and scope

`const` — cannot be reassigned. `let` — can be reassigned. `var` is the 1995 original; it still works but is considered obsolete because it doesn't respect block scope the way `let` / `const` do — avoid it entirely.

```
const title = "Draft post";
let count = 0;
count = count + 1;   // fine — let allows reassignment
title = "New title"; // TypeError: Assignment to constant variable
```

Both are **block-scoped**: a variable declared inside `{ }` — an `if`, a `for`, any bare block — does not exist outside that block. Default to `const` everywhere; reach for `let` only when a value genuinely needs to change later.

One nuance worth knowing up front: `const` locks the *variable binding*, not the contents of an object it points to:

```
const user = { name: "Alice" };
user.name = "Bob";        // fine — not reassigning `user`, just changing what it points to
user = { name: "Carol" }; // TypeError — this WOULD reassign the binding
```

0.4 — Hoisting and the temporal dead zone

One term correction first: the actual JS name is **temporal dead zone (TDZ)**, not "deadlock" — deadlock is an unrelated concept from concurrent programming (two processes each stuck waiting on the other). Different problem, worth not conflating when searching documentation.

Before running any code, JS does a pass through the current scope and registers every declared name first — as if every declaration were quietly lifted to the top of its scope. That pre-registration is **hoisting**.

Function declarations are hoisted completely — name and body both, callable even above their own definition:

```
sayHi(); // works fine, runs before the definition below

function sayHi() {
  console.log("hi");
}
```

`var` is hoisted but only the name, not the value — using it early gives `undefined` rather than an error:

```
console.log(x); // undefined — not a crash, but not 5 either

var x = 5;
```

`let` / `const` are hoisted too, but locked behind the TDZ — touching them before their own declaration line runs throws an actual error instead of silently giving `undefined`:

```
console.log(y); // ReferenceError: Cannot access 'y' before initialization
let y = 5;
```

Practical rule: always define a variable before using it, regardless of what hoisting technically allows. This is a "why does this specific error say what it says" tool, not something to lean on stylistically.

0.5 — undefined, null, and truthy/falsy

`undefined` is JS's automatic absence — a variable declared but never assigned, a missing object property. `null` is different: a value a human deliberately assigned to mean "empty on purpose." JS never produces `null` on its own.

JS allows any value, not just booleans, inside an `if` condition, converting it internally. Exactly eight values are falsy: `false, 0, -0, 0n, "", null, undefined, NaN`. Everything else is truthy — **including empty arrays and empty objects**:

```
if ([]) console.log("runs");  // yes — an empty array is truthy in JS
if ({}) console.log("runs");  // yes — so is an empty object
if (0) console.log("nope");
if ("") console.log("nope");
```

`?.` (optional chaining) uses this: it checks whether the left side is null/undefined first, returning `undefined` instead of throwing if so — `session?.user` is safe even when `session` is `null`.

0.6 — Type conversion

Explicit — you ask for the conversion directly:

```
Number("42")     // 42
String(42)       // "42"
Boolean(0)       // false
parseInt("42px") // 42 — stops at the first non-digit character
```

Implicit (coercion) — JS converts on its own, and this is where real bugs live:

```
"5" + 3   // "53"  — + next to a string means concatenate; the number becomes a string
"5" - 3   // 2     — -  has no string meaning, so the string becomes a number instead
"5" == 5  // true  — == allows coercion before comparing
"5" === 5 // false — === compares type AND value, no coercion at all
```

RULE OF THUMB Always use `===` / `!==`, never `==` / `!=`. The loose versions' coercion rules are a well-documented source of bugs; the strict versions behave the way you'd actually expect.

0.7 — Control flow: if/else

```
if (score > 90) {
  grade = "A";
} else if (score > 75) {
  grade = "B";
} else {
  grade = "C";
}

// ternary — a compact if/else that PRODUCES a value directly
const grade = score > 90 ? "A" : "C";

// switch — many exact-match branches
switch (status) {
  case "done":  label = "Complete"; break;
  case "error": label = "Failed"; break;
  default:      label = "Pending";
}
```

Mechanically identical to R's/Python's `if` / `else`. The one real gotcha: the truthy/falsy conversion rules above govern what counts as "true" inside that condition — `if (count)` is false when `count` is `0`, which surprises people coming from stricter languages.

0.8 — Functions: declaring and calling

Declare once, call as many times as needed by writing the name followed by parentheses:

```
function greet(name, greeting = "Hello") { // "greeting" has a default value
  return `${greeting}, ${name}!`;
}

greet("Vignesh");            // "Hello, Vignesh!" — greeting uses its default
greet("Vignesh", "Welcome"); // "Welcome, Vignesh!"
```

`name` and `greeting` in the definition are **parameters** — placeholders. `"Vignesh"` and `"Welcome"` at the call site are **arguments** — the actual values passed in. "Calling" a function always means writing its name immediately followed by `( )` with any arguments inside; without the parentheses, `greet` just refers to the function itself — it does not run it.

0.9 — Arrow functions: complete reference

A second, more compact way to write a function, in near-universal use in modern JS. Every

syntax variant:

```
() => expression                        // no parameters
(x) => expression                       // one parameter (parens preferred — style rule below)
(x, y) => expression                    // multiple parameters — parens required, no exception
(x) => { const y = x * 2; return y; }  // block body — braces AND explicit return required
(x = 10) => x                           // default parameter
(...args) => args.length                // rest parameter
({ name, id }) => `${name}-${id}`       // destructured parameter
```

The gotcha everyone hits once: returning an object literal directly needs wrapping parens — `{` immediately after `=>` is parsed as the start of a code block, not an object:

```
const toOption = (id, label) => ({ id, label }); // correct — parens mark this as an expressi
```

Where arrow functions belong: array method callbacks (`.map` / `.filter` / `.forEach` — the large majority of real-world use), Promise chains (`.then((data) => ...)`), and any callback where you want the surrounding code's `this` to carry through unchanged, since arrow functions never create their own `this` — more on that in 0.11.

Where they don't belong: object methods that need `this` to refer to the object itself (use regular method syntax), constructors (arrow functions cannot be used with `new` — throws a TypeError), and anywhere the `arguments` object is needed — arrows have no `arguments` of their own; a rest parameter (`...args`) covers the same need in both regular and arrow functions.

STYLE GUIDE

The **Airbnb JavaScript Style Guide** — the de facto community standard most companies' ESLint configs are built on — requires parens around every parameter, even a single one (`(x) => x` , not `x => x`), so adding a second parameter later never changes the visual shape of the code. It also limits the brace-free, implicit-return form to short, single-expression, side-effect-free transformations; once logic needs more than one statement, use the block-body form with an explicit `return` instead of cramming it into one line.

0.10 — Closures

An inner function keeps a live, working reference to the variables from the scope it was created in, even after that outer scope has already finished running:

```
function makeCounter() {
  let count = 0;
  return function() {
    count += 1;
    return count;
  };
}
const counterA = makeCounter();
const counterB = makeCounter();
counterA(); // 1
counterA(); // 2 — counterA's own private count kept incrementing
```

```
counterB(); // 1 — counterB has a SEPARATE count, untouched by counterA
```

Each call to `makeCounter()` creates a brand-new `count` and a brand-new inner function closing over that specific variable — this is the standard way JS gets private, per-instance state without needing a class.

0.11 — The this keyword

Inside a regular `function`, `this` is decided by *how the function is called*, not where it's written — a frequent source of bugs:

```
const btn = { label: "Send", handleClick: function() { console.log(this.label); } };
document.querySelector("button").addEventListener("click", btn.handleClick);
// logs undefined — `this` is decided by the CALLER (the button), not by btn
```

An arrow function has no `this` of its own — it simply uses whatever `this` was in the surrounding code at the point it was written, which is exactly why arrow functions are the default choice for callbacks and event listeners: nobody has to reason about `this` binding at all.

0.12 — Classes

A template for creating objects that share the same shape and behavior. `constructor` runs once, automatically, when a new instance is created with `new`:

```
class ChatMessage {
  constructor(text, role) {
    this.text = text;
    this.role = role;
  }
  isFromUser() {
    return this.role === "user";
  }
}

const msg = new ChatMessage("hi there", "user"); // creates an instance
msg.isFromUser(); // true — calling a method, same () syntax as any function
```

PYTHON Directly parallel to Python's `class` / `__init__` / `self` — JS's `constructor` is `__init__`, and `this` inside a class behaves the way Python's explicit `self` does (methods called normally, not as standalone callbacks, keep `this` predictable — the ambiguity in 0.11 shows up specifically when a method gets detached and passed around as a callback).

0.13 — What you actually need to carry forward

- Default to `const`; reach for `let` only when reassigning; never write `var`.

- Objects/arrays copy by reference, not by value — mutating one variable can mutate what another variable points to.
- Always use `===` / `!==`, never `==` / `!=`.
- `typeof null` lies — it says `"object"`.
- Empty arrays/objects are truthy in JS — the opposite of Python.
- Arrow functions for callbacks; regular functions/methods when `this` needs to mean the object itself.
- When unsure of a method or API, MDN (developer.mozilla.org) is the reference, not a guess.

Part 2 — jQuery for Shiny

Not a detour — Shiny's own JS core (`shiny.js`) was built on jQuery for years, and a lot of Shiny extension packages (shinydashboard, older custom widgets) still write `$(...)` instead of `document.querySelector(...)`. You'll meet this syntax reading other people's Shiny JS even where your own code stays vanilla.

0.14 — What jQuery is

A JavaScript library that wraps DOM operations in a shorter, more forgiving syntax. Where vanilla JS needs several lines to select an element and check it exists before acting on it, jQuery collapses that into one chain. It has to be loaded before you can use it — a `<script>` tag pointing at the jQuery file, placed before your own code runs:

```
<script src="https://code.jquery.com/jquery-3.5.1.js"></script>
```

Version matters for Shiny specifically: match whatever jQuery version Shiny itself ships, so you're not running two different copies of the library at once.

0.15 — Syntax: the one pattern everything follows

```
$(selector).action();
```

`$` is just a function name — jQuery's entire public interface is one function, conventionally aliased to the dollar sign. `selector` uses the exact same CSS selector syntax as `querySelector` (class, id, element, attribute). `action()` is whatever you want done — read a value, change a style, attach a listener.

```
// vanilla JS — has to null-check before reading a property
let el = document.querySelector('.text');
let inner = el ? el.innerHTML : undefined;

// jQuery — same result, no manual null-check needed
let inner = $('.text').html();
```

That null-check difference is the actual reason jQuery caught on: `$('.text')` never throws even if nothing matches — it returns an empty jQuery object you can safely keep chaining off of, where vanilla `querySelector` returns `null` and crashes on the next line if you don't check first.

0.16 — Traveling the DOM

jQuery's traversal methods, the ones that come up constantly in real Shiny code:

```
children()   // direct children only, one level down
find()       // descendants at ANY depth (like querySelectorAll, but from this element)
closest()    // nearest ancestor (including itself) matching a selector — travels UP
siblings()   // all siblings of the matched element(s)
next()       // the immediately following sibling
prev()       // the immediately preceding sibling
first()      // first item in a matched set
last()       // last item in a matched set
filter()     // keep only elements matching a condition
not()        // exclude elements matching a condition
```

`closest()` is the one worth internalizing first — it's the jQuery equivalent of vanilla's `element.closest(selector)` (which vanilla JS actually borrowed from jQuery), used constantly for "find the card/row/container this button lives inside" style lookups.

0.17 — Manipulating tags

```
addClass('name')      // add a class
removeClass('name')   // remove a class
hasClass('name')      // check if a class is present — returns true/false
attr('name')          // get an attribute; attr('name', 'val') sets it
css('prop')           // get a CSS property; css('prop', 'val') sets it
val()                 // get a form element's current value
remove()               // delete the element from the DOM entirely
after() / before()    // insert content immediately after / before
```

```
$('.status-badge').addClass('active').css('color', 'green');
```

0.18 — Chaining

Since most jQuery methods return the same jQuery object they were called on, you can call another method directly on the result — no need to re-select the element each time:

```
$('ul')
  .first()
  .css('color', 'green')
  .attr('id', 'myAwesomeItem')
```

```
  .addClass('amazing-ul');
```

Four operations on the same element, one statement, ending in a single `;`. Convention is one method per line, indented, so the chain reads top to bottom.

0.19 — Events

```
$('#btn').on('click', function() {
  // handle the click
});

// shorthand for common events also exists
$('#btn').click();     // triggers a click programmatically
$('#input').change();  // triggers a change event

// .on() is what you reach for with CUSTOM, non-standard event names —
// this is how shinydashboard and similar packages build their own event systems
$(document).on('shiny:connected', function() {
  // fires on a Shiny-specific custom event, not a native browser event
});
```

`.on()` is the one to know well — unlike `addEventListener`, it accepts event names that don't exist natively in the browser at all, as long as something else in the page calls `.trigger('that-name')` to fire it. That's the mechanism most Shiny extension packages use to build their own internal event systems on top of jQuery.

0.20 — The document-ready pattern

Running jQuery code before the page has finished loading means selectors find nothing — the elements don't exist yet. The standard guard:

```
$(document).ready(function() {
  // your code — guaranteed the DOM is fully loaded
});

// shorthand for the exact same thing
$(function() {
  // your code
});
```

Skipping this is the single most common source of "why is my jQuery code doing nothing" bugs — the selector silently matches zero elements because it ran too early, and nothing errors, it just quietly does nothing.

0.21 — What you need to carry forward (jQuery)

- `$(selector).action()` is the entire pattern — everything else is which selector, which action.

- jQuery selectors never throw on zero matches — vanilla `querySelector` returns `null` and needs a manual check first.
- `closest()`, `find()`, `siblings()` cover most real traversal needs.
- `.on()` handles both native events and Shiny's own custom event names — `addEventListener` only handles native ones.
- Always wrap jQuery code in `$(document).ready(...)` or its `$(function(){...})` shorthand.

MODULE 1 · CORE LANGUAGE FOUNDATIONS

Lesson 1 — Variables & Values

const/let, block scope, primitive vs. reference values, and why const doesn't freeze an object.

1.1 — const, let, and why var is dead

Three ways to declare a variable. `var` is the 1995 original — function-scoped, causes bugs, you will never write it. `let` and `const` are block-scoped: they only exist inside the `{ }` they were declared in.

```
const title = "Draft post";   // cannot be reassigned
let count = 0;                 // can be reassigned
count = count + 1;             // fine
title = "New title";           // TypeError: Assignment to constant variable
```

R Same idea as R's `{ }` blocks in `if` / `for`, except JS enforces the boundary strictly rather than leaking the variable out.

PYTHON Python has **no block scope at all** — a variable assigned inside an `if` or `for` is still visible after it ends. JS's block scoping is actually stricter than both R and Python here.

Default to `const`. Only reach for `let` when the variable will genuinely change.

1.2 — Primitive vs. reference values

The concept most likely to trip you coming from R, because R copies almost everything by value. JS does not.

Primitives (string, number, boolean, undefined, null) are copied by value:

```
let a = 5;
let b = a;   // b gets a COPY of 5
b = 10;
console.log(a); // still 5 — a and b are independent
```

Objects and arrays are copied by reference — the variable holds a pointer to shared data, not the data itself:

```
let obj1 = { name: "draft" };
let obj2 = obj1;         // obj2 points to the SAME object as obj1
obj2.name = "published";
console.log(obj1.name);  // "published" — same object, both changed
```

This is exactly why React's rule is "never mutate state directly, always create a new object." Same reference, same memory address — React can't tell anything changed, so it won't re-render.

PYTHON Python's mutable vs. immutable split is the closer match, even than R: `list2 = list1` aliases the same list — `list2.append(4)` shows up in `list1` too, identical to the JS array behavior above. `int` / `str` / `tuple` copy by value, like JS primitives.

1.3 — const does not freeze the object

`const` locks the *variable binding*, not the *contents*:

```
const user = { name: "Alice" };
user.name = "Bob";        // fine — not reassigning `user`, just mutating what it points to
user = { name: "Carol" }; // TypeError — this reassigns the binding, which const forbids
```

"Same box, two labels": `const arr2 = arr1` doesn't copy the array — `arr2.push(4)` mutates the one shared box, so `arr1` changes too.

1.4 — The 7 basic value types

JS has exactly 7 **primitive** types, plus one catch-all non-primitive type. `typeof` tells you which one you're holding:

```
typeof "hello"     // "string"
typeof 42            // "number"
typeof true            // "boolean"
typeof undefined         // "undefined"
typeof null                // "object" — a 25-year-old bug; null is NOT actually an object
typeof 10n                   // "bigint"
typeof Symbol()                // "symbol"
typeof {}                        // "object"
```

string, number, boolean, undefined, null, bigint, symbol — those 7. Everything else is an **object** (plain objects, arrays, functions, dates) — the reference-type category above, copied by pointer, not by value. `bigint`'s `n` suffix marks arbitrary-precision integers past `Number.MAX_SAFE_INTEGER`; you'll rarely write one by hand.

PYTHON Python's `type(x)` is the `typeof` equivalent. One real difference: Python keeps `int` and

`float` as separate types, where JS has one `number` for both — `42` and `42.0` are the exact same JS value.

▸ **Check yourself**

MODULE 1 · CORE LANGUAGE FOUNDATIONS

Lesson 2 — undefined/null & Truthy-Falsy

Two flavors of "nothing," optional chaining, and where JS quietly differs from Python.

2.1 — undefined vs. null

`undefined` is JS's default absence — a variable declared but never assigned, a function with no return, a missing property:

```
let x;
console.log(x); // undefined — declared, never assigned

const obj = { name: "Alice" };
console.log(obj.age); // undefined — no such property, no crash
```

`null` is different: a value someone deliberately assigned to mean "empty on purpose." JS never hands you null unprompted.

R `null` maps roughly to R's `NULL`. `undefined` has no clean R equivalent — and don't reach for R's `NA` here; that's about missing data in a vector, a different problem.

PYTHON Python collapses this whole distinction into one concept, `None`. There's no Python equivalent of "declared but never assigned" producing a distinct value the way JS's `undefined` does.

2.2 — Optional chaining ?.

Without it, reaching into a possibly-missing object throws and kills the program. `?.` checks first — if the left side is null/undefined, stop and return undefined instead of crashing.

```
const session = null;
console.log(session.user);  // TypeError: Cannot read properties of null
console.log(session?.user); // undefined — no crash
```

2.3 — Truthy/falsy conversion

JS lets you use *any* value in an `if`, converting it internally. Exactly eight values are falsy: `false, 0, -0, 0n, "", null, undefined, NaN`. Everything else is truthy — **including `[]` and `{}`**.

```
if ([]) console.log("runs");   // yes — empty array is truthy in JS
if ({}) console.log("runs");   // yes — empty object too
if (0) console.log("nope");

if ("") console.log("nope");
```

PYTHON — OPPOSITE BEHAVIOR

In Python, `if []:` is `False` — empty containers are falsy. In JS, `if ([])` **runs** — empty arrays and objects are truthy. This is a direct reversal between the two languages, not just a missing feature, and it trips people constantly.

Real guard clause, read right to left: `if (!session?.user) throw new Error('Unauthorized')` — get user safely, or undefined if session itself is null; `!` flips it. One line covers "no session" AND "session with no user."

▸ **Check yourself**

MODULE 1 · CORE LANGUAGE FOUNDATIONS

Lesson 3 — Functions

Declarations vs. arrows, hoisting, closures, this binding, block scope, higher-order functions, naming conventions, and a complete arrow-function reference — pulled from a real Shiny custom-JS file.

3.1 — Declarations vs. arrow functions

Named `function` for reusable utilities called by name elsewhere. Arrow functions for one-off callbacks handed straight to something else:

```
function novaTextarea() {
  return document.querySelector(NOVA_SEL.textarea);
}

document.addEventListener("click", (ev) => {
  const btn = ev.target.closest(".nova-card-actions button");
});
```

3.2 — Hoisting

Before running any code, JS registers every declared name in the current scope first. Function declarations are fully hoisted — name AND body, usable above their own definition:

```
sayHi(); // works fine

function sayHi() { console.log("hi"); }
```

`let` / `const` are hoisted but locked behind a "temporal dead zone" — using them before their line runs throws instead of silently giving undefined:

```
console.log(y); // ReferenceError: Cannot access 'y' before initialization
let y = 5;
```

R R has nothing like this — it evaluates and executes as it goes, no separate registration pass.

PYTHON Python has no hoisting either — a name must be defined above the line that uses it, full stop. That's actually closer to JS's `let` / `const` behavior (the temporal dead zone) than to JS's function-hoisting, which has no real Python equivalent.

3.3 — Closures

An inner function keeps a live reference to variables from the scope it was born in, even after that scope has finished running:

```
novaAttachments.forEach((att) => {
  remove.addEventListener("click", () => {
    const i = novaAttachments.indexOf(att); // att still correct, minutes later
    if (i >= 0) novaAttachments.splice(i, 1);
  });
});
```

Using an index instead of closing over `att` directly would break the moment an earlier chip is removed and every later index shifts.

PYTHON Python closures work the same way — a nested `def` captures its enclosing scope. One real gap: *reading* an outer variable needs nothing special, but *reassigning* one from inside the inner function requires the `nonlocal` keyword. JS has no such requirement either way.

3.4 — Arrow functions and this

A regular function gets a fresh `this`, decided by how it's *called*. An arrow function has none of its own — it uses whatever `this` was in the surrounding code when written. Why nearly every listener in real Shiny JS is an arrow function.

PYTHON Python sidesteps this entire problem — `self` is just an explicit first parameter you name

PYTHON Python sidesteps this entire problem: `self` is just an explicit first parameter you name yourself — never inferred from how the method was called, so none of this ambiguity exists.

3.5 — Bare block scope

A stray `{ }` with no function around it is just a scope wall — keeps a variable from leaking as a global. Pre-2015, people got the same effect with an IIFE: `(function(){ ... })()`.

3.6 — Higher-order functions

Functions that take or return functions — the array methods everywhere in real code:

```
kids.every((k) => k.classList.contains("nova-tool-line"))
```

R Same shape as `purrr::every()` or `all(sapply(kids, f))` — different syntax, identical idea.

PYTHON `.every()` maps to Python's built-in `all()`. `.map()` / `.filter()` have direct Python equivalents too (`map()`, `filter()`), but idiomatic Python usually reaches for a list comprehension instead — `[k for k in kids if cond]` rather than chaining `.filter()`.

3.7 — Naming conventions

Not enforced by the language, followed anyway. **camelCase** — variables, functions, methods (default for almost everything). **PascalCase** — classes/constructors: `Array`, `Promise`, `MutationObserver`. **UPPER_SNAKE_CASE** — constants meant as fixed configuration, a human signal ("don't edit this casually"), not enforcement.

PYTHON — THE ONE TO WATCH PEP 8 uses `snake_case` for variables and functions — the opposite of JS's camelCase default. Classes (`PascalCase`) and constants (`UPPER_SNAKE_CASE`) actually match JS convention. The variable/function case is the one that will keep autopiloting wrong when you switch languages mid-day.

3.8 — Arrow functions: complete reference

Expanding on 3.1 — the full syntax, every variant:

```
() => expression                        // no parameters
(x) => expression                       // one parameter (parens preferred — style rule below)
(x, y) => expression                    // multiple parameters — parens required
(x) => { const y = x * 2; return y; }  // block body — braces AND explicit return required
(x = 10) => x                           // default parameter
(...args) => args.length                // rest parameter
({ name, id }) => `${name}-${id}`       // destructured parameter
```

The gotcha everyone hits once: returning an object literal directly needs wrapping parens

— `{` right after `=>` parses as a block, not an object:

```
const toOption = (id, label) => ({ id, label }); // correct — parens mark this as an expressi
```

Where they belong: array method callbacks (`.map` / `.filter` / `.forEach` — the overwhelming majority of real-world use), Promise chains (`.then((data) => ...)`), and event listeners — because an arrow function never creates its own `this` , it simply uses whatever `this` was in the surrounding code, which is exactly the behavior a listener callback wants.

Where they don't: object methods needing `this` to mean the object itself (use regular method syntax instead), constructors (arrow functions can't take `new` — throws a TypeError), and anywhere the `arguments` object is needed (arrows have none of their own — use a rest parameter).

STYLE GUIDE The **Airbnb JavaScript Style Guide** — the de facto community standard most companies' ESLint configs are built on — requires parens around every parameter, even a single one (`(x) => x` , not `x => x`), so adding a second parameter later never changes the visual shape. It also limits implicit-return (no-brace) form to short, single-expression, side-effect-free transformations — once logic needs more than one statement, use the block-body form with an explicit `return` rather than cramming it into one line.

Bonus — document & querySelector

`document` is a global the *browser* creates automatically on page load — not imported, not a module. It's really `window.document` . Browser-only: plain Node has no `document` at all.

`querySelector` is a *method* on document (or any element) taking a CSS-selector string, returning the first match or null:

```
document.querySelector("shiny-chat-container textarea");
```

Type `document.` in DevTools' Console, or in a `.ts` file in your editor (TypeScript ships `lib.dom.d.ts`) for full autocomplete — the closest thing to R's `dir()` /autocomplete here. Reference: **developer.mozilla.org/en-US/docs/Web/API/Document**.

▸
Check yourself

MODULE 5 · SHINY INTEGRATION API

Lesson 21 — Shiny.setInputValue

Why R and JS need a persistent connection at all, and how JS sends a value back to the server.

21.1 — Why a plain HTTP request isn't enough

An ordinary web request closes the connection as soon as it's answered — fine for loading a page once, but not for an app where the server needs to react every time you move a slider or click a button, potentially dozens of times a minute. Shiny instead opens one persistent, two-way connection per browser tab — a **WebSocket** — the moment the app loads, and keeps it open for the life of the session. Both sides can send messages down that same connection at any time, in either direction, without re-opening anything.

One consequence worth knowing: R itself is single-threaded, so if two people have the same app open, their messages queue and get handled one at a time on the server side — not a bug, just how a single R process necessarily works.

21.2 — Sending a value from JS to R

`Shiny.setInputValue(id, value)` is the JS-side call that pushes a value down that WebSocket connection to the server, where it shows up as `input$id` — exactly as if a built-in input widget had produced it.

```
document.querySelector('#theme-toggle').addEventListener('click', () => {
  const isDark = document.body.classList.toggle('dark-mode');
  Shiny.setInputValue('current_theme', isDark ? 'dark' : 'light');
});
```

On the R side, that value is just `input$current_theme`, reactive like any other input — an `observeEvent(input$current_theme, {...})` fires every time this line runs.

21.3 — The priority option

By default, Shiny only triggers reactivity when the value actually *changes* — setting the same value twice in a row does nothing the second time. Passing a third argument overrides that:

```
Shiny.setInputValue('current_theme', 'dark', {priority: 'event'});
```

With `{priority: 'event'}`, every call fires the reactive, even if the value is identical to last time — needed for things like a "retry" click where the value itself doesn't change but the action should still re-run.

MODULE 5 · SHINY INTEGRATION API

Lesson 22 — addCustomMessageHandler

The reverse direction: R pushing data to the browser without waiting for the user to do anything.

22.1 — Why this is a separate mechanism from setInputValue

`Shiny.setInputValue` (Lesson 21) only moves data JS → R. For R → JS — the server deciding, on its own, to tell the browser something (a background job finished, a scheduled refresh happened) — Shiny pairs two calls, one on each side, matched by a shared string name:

- **R side:** `session$sendCustomMessage(type, message)`
- **JS side:** `Shiny.addCustomMessageHandler(type, function(message) {...})`

The `type` string is what links them — think of it as a named channel. R sends on that channel, JS listens on that same channel; the two sides never call each other directly.

22.2 — A worked example

R side, inside the server function:

```
observeEvent(input$refresh_data, {
  new_count <- nrow(get_latest_rows())
  session$sendCustomMessage("row-count-updated", list(count = new_count))
})
```

JS side, registered once when the page loads:

```
Shiny.addCustomMessageHandler('row-count-updated', function(message) {
  document.querySelector('#row-count-badge').textContent = message.count;
});
```

`message` arrives in JS as a plain object — whatever R's `list(...)` contained gets converted to JSON automatically on the way over, so `message.count` just works without any manual parsing.

22.3 — Registration timing matters

`addCustomMessageHandler` has to run *before* R ever sends on that channel, or the message arrives with nothing listening and is simply dropped — no error, just silently lost. That's why it's typically registered inside a document-ready block (Lesson 0.20) right when the custom JS file loads, not inside some later conditional path that might not have run yet.

MODULE

Lesson

NOT WRITTEN YET

This page is on the syllabus but hasn't been taught yet.

Ask for it by name in chat and it lands here next.